LITHAVEN CAMPUS STUDY VAULT • TOPPER-VERIFIED REVISION CAPSULE

Machine Learning & Deep Learning Mathematics Formula Capsule

Subject: Machine Learning & Mathematical Foundations | Author: Dr. Marcus Vance (AIIMS / Johns Hopkins) | 100% Free Campus Access

1. LOSS FUNCTIONS & DERIVATIVES

• **Mean Squared Error (MSE):** L(y, y_hat) = (1/N) * sum((y_i - y_hat_i)^2). Gradient with respect to weight: dL/dw = (-2/N) * sum(x_i * (y_i - y_hat_i)). Convex surface with unique global minimum.
• **Binary Cross-Entropy (Log Loss):** L(y, p) = - [y * log(p) + (1 - y) * log(1 - p)]. Used with Sigmoid activation. Derivative with respect to logit z = p - y.
• **Categorical Cross-Entropy:** L(y, p) = - sum_k (y_k * log(p_k)). Paired with Softmax activation: p_i = exp(z_i) / sum(exp(z_j)). Derivative dL/dz_i = p_i - y_i.

2. OPTIMIZATION ALGORITHMS

• **Stochastic Gradient Descent (SGD) with Momentum:** v_t = beta * v_{t-1} + (1 - beta) * grad_t; theta_t = theta_{t-1} - alpha * v_t. Damps oscillations along high-curvature directions.
• **Adam Optimizer:** Computes adaptive learning rates using first moment m_t (mean) and second raw moment v_t (uncentered variance). Bias correction: m_hat = m_t / (1 - beta1^t), v_hat = v_t / (1 - beta2^t). Update: theta = theta - (alpha / (sqrt(v_hat) + eps)) * m_hat.

3. TRANSFORMER ATTENTION MECHANISMS

• **Scaled Dot-Product Attention:** Attention(Q, K, V) = Softmax((Q * K^T) / sqrt(d_k)) * V. The scaling factor 1/sqrt(d_k) prevents extremely small softmax gradients in high dimensional spaces.
• **Multi-Head Attention (MHA):** MultiHead(Q, K, V) = Concat(head_1, ..., head_h) * W_O, where head_i = Attention(Q * W_Q_i, K * W_K_i, V * W_V_i). Allows model to jointly attend to info at different positions.

4. REGULARIZATION & GENERALIZATION

• **L1 Regularization (Lasso):** Adds lambda * sum(|w_i|). Drives non-essential weights to strictly zero, producing sparse models and automatic feature selection.
• **L2 Regularization (Ridge / Weight Decay):** Adds (1/2) * lambda * sum(w_i^2). Pulls weights towards zero without making them exactly zero. Prevents individual features from dominating decisions.
• **Batch Normalization (BN):** Normalizes mini-batch activations: x_hat = (x - mu_B) / sqrt(sigma_B^2 + eps). Scales and shifts: y = gamma * x_hat + beta. Accelerates convergence and stabilizes deep networks.